

The First Mandatory Programming Assignment:

The Java-RMI International Statistics Service

IN5020/9020 Autumn 2026

Objectives

1. To develop an object-based distributed system;
2. Using the Java-RMI programming model and remote method invocation;
3. Following the simple client/server architecture with a load balancer;
4. Handling large data sets in a distributed environment;
5. Simulating client-server remote communication on a local machine;
6. Implementing caching mechanisms as a distributed scaling technique;
7. Implementing Docker/Container to host servers.

Scope

The system consists of a distributed International Statistics Service. The Application functionality is provided by a remote object residing at the server side. Client objects interact with the server through remote method invocations. The client can invoke the methods defined in the server's remote interface specification.

The system consists of a load balancing server, and several servers that have access to an International Statistics database described below.

The load balancing server acts as a proxy and is in charge of distributing users' requests to other servers. **Clients make a remote method call to the proxy server and it replies with the address and port number of one of the servers to clients. Clients then invoke a remote call to the server that got their address and port and send their request.**

We will discuss the technical details of the proxy server and other servers further in this document.

The servers will have access to a database consisting of the statistics dataset. The dataset is available for download from the course website in a single zip archive named *Ass1-RMI.zip*. The dataset contains information about 140,574 different cities in the world. For more information about the datasets refer the following website: <https://public.opendatasoft.com/explore/dataset/geonames-all-cities-with-a-population-1000/>. The sample representation of the dataset is given below:

Geoname ID	Name	Country Code	Country name EN	Population	Timezone	Coordinates
2798301	Fleron	BE	Belgium	15994	Europe/Brussels	50.61516,5.68062
2798438	Fauvillers	BE	Belgium	1952	Europe/Brussels	49.85116,5.66405
2798581	Ettelgem	BE	Belgium	1181	Europe/Brussels	51.17984,3.02919
2798615	Essen	BE	Belgium	10000	Europe/Brussels	51.46791,4.46901
2798680	Erondegem	BE	Belgium	1742	Europe/Brussels	50.94118,3.95611
2798868	Eksaarde	BE	Belgium	6005	Europe/Brussels	51.14876,3.96814
2798949	Eghezee	BE	Belgium	14352	Europe/Brussels	50.59076,4.91175

The service you will implement is responsible for parsing the dataset and providing clients with statistics information about different countries. The clients, using remote invocations, can ask the query server for different information it requires.

Technical features

1. Proxy (load balancing) Server

The proxy server distributes the requests based on the zone that each client is located in. To simulate geographical zones in our assignment, each query will be assigned a zone number. We will discuss the format of queries in the next subsection.

We have several different geographical zones in our scenario and one server with access to the database in each zone. **The proxy server will try to connect a client in a specific zone to the server in the same zone. If the server in that specific zone is overloaded with client requests (equal or more than 18 requests in the waiting list), the load balancing server tries servers in the other zones to find a server with the least requests in the waiting list. The priority for establishing a communication to the neighbor zone server will be the server with fewer requests in the waiting list. If several servers have the same minimum number of requests in the waiting list, the server closest to the client (clockwise) will be chosen. If all servers are also overloaded (more than 18 requests in the waiting list) the server in the same zone will be assigned for the user request.**

If a Zone does not contain a server, the proxy will automatically change the target zone of the client request to the closest one (clockwise) and repeat the steps above. For example, if only Zone 1, 3, 4, 5, 7 has server in it, the client requests from Zone 6 will be automatically changed to 7, and client requests from Zone 8 will be automatically changed to Zone 1 by proxy server. However, the results are still returned to clients 6 and 8 respectively.

Figure 1 illustrates how the proxy server acts in the system and how different geographical zones are connected as neighbors, given that there are 5 zones.

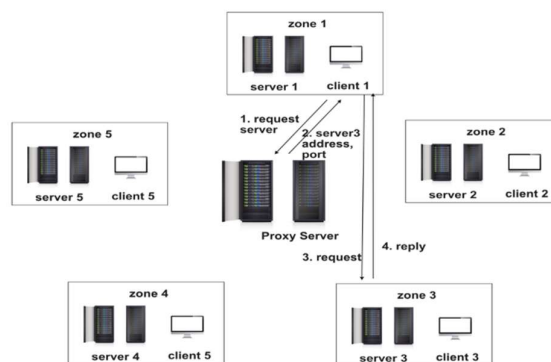


Figure 1. Proxy Server and Neighbor Zones

Figure 1 shows a client in zone 1 that asks for a connection from the proxy server. Proxy server will first check the server in the same zone and if it not overloaded (less than 18 requests in the waiting list), it will return to the client the address and port to the server in the same zone so that the client can send the request to the server and get the response (Scenario A as indexed by 3.A and 4.A) in the figure. On the other hand, if the server in the same zone is overloaded, the proxy server will check the other servers (Zones 2 & 3 & 4 & 5) and return the address of nearest server (clockwise) that is not overloaded (or the server at same zone if all servers are also overloaded).

You can decide on the design of the remote interface of the proxy server for both clients and servers on different zones. **Proxy needs to provide a remote interface for clients so that clients can invoke its methods.** Additionally, **proxy needs access to servers' remote interface to fetch information about servers' load and waiting channel every once in a while.** *After every 18 requests that the proxy assigns to a specific server (18 replies to clients with the same server address), it will invoke a remote call to that server to fetch updated information about its workload and number of tasks in the waiting list. The proxy will have independent counter for each server.* Hence, *fetching information from each server will take place only after assigning that specific server to clients every 18 times.* You also have to consider that updating the proxy with servers' workload should not interrupt servers' or proxy's execution mode and it should take place in another thread.

The Proxy server should have an API so that new servers can invoke in order to register. The Proxy server then assigns Zone numbers in ascending order and records the server's IP address and port.

In real distributed systems, establishing a communication to different servers will take different amount of time. In order to simulate the connection time in this assignment, you should consider that communication to the server in the same zone takes 80 milliseconds and communication to servers in a neighbor zone will take $80 + X * 30$ milliseconds, which X is the difference in Zone values. E.g Client Zone 1 sends to Zone 3 will need a $80 + (3-1) * 30 = 140$ milliseconds. More details on how to simulate the communication time will be presented in the next subsection.

2 Server

1. The service interface exposes the following methods that have to be implemented by a servant class
 - a. **getPopulationofCountry(countryName)**
 - i. given a country name as input, return the population of the country by summing up the population of cities in that country
 - ii. eg:- country name = "Norway" => population = 3,162,856
 - iii. eg:- country code = "Sweden" => population = 9,362,428
 - b. **getNumberofCities(countryName, threshold, comp)**
 - i. given a country name, threshold and comp as input, return the total number of cities in the given country that contains at least/most the "threshold" amount of population.

- ii. comp is either “min”, or “max”.
- iii. eg:- country code = “Norway”, threshold = “100,000”, comp= “min”
=> result = 4

c. getNumberOfCountries(citycount, threshold, comp)

- i. Return the number of countries that contain at least ‘citycount’ number of cities where each city has the population of at least/most ‘threshold’ number of people.
- ii. comp is either “min”, or “max”.
- iii. eg:- city count=2; threshold= 5,000,000; comp= “min” => result = 7

d. getNumberOfCountriesMM(citycount, minpopulation, maxpopulation)

- i. Return the number of countries that contain atleast ‘citycount’ number of cities where each city has population between ‘minpopulation’ and ‘maxpopulation’.
- ii. eg:- city count=30, min population= 100,000, max population = 800,000 => result = 30

2. Requirements

- a. The stubs of the remote objects (instance of the server class) should be instantiated and registered with unique names on different ports (anonymous ports can also be used).
 - b. (For Group of 4) You should create an image of docker to containerize the code as well as other necessary libraries tools. Then you can run a container to simulate a server.
 - c. (For Group of 3) You can create a ServerSimulator class that will create 5 instances of the server class in different addresses and ports.
 - d. A simple example will be discussed on the group meeting
3. In a distributed environment there is always communication latency. To simulate network latency, **pause the server execution for 80 milliseconds every time a remote method is invoked, before adding to queuing list**. This way, you can test your code by deploying both client and server in the same machine.
- a. If a server in the other zone is assigned to the client, the total communication time will increase to 80+X*30 milliseconds.
4. Each of the servers in different geographical zones maintain their own zone-specific waiting list. If the server is already in the execution mode, the new incoming task will be added to the waiting list.
- a. The servers have a FIFO policy for executing the client requests in the waiting list.
 - b. Adding new incoming tasks to the waiting list should not interfere and be interfered due to server being in execution mode.
 - c. You can implement different threads to isolate the waiting list and execution mode of servers. However, the server execution (processing requests) will take place in one single thread.
 - i. In other words, each server will have 2 groups of threads. One group has only one thread, for executing the tasks on top of the waiting list.

The other group may have multiple threads to accept new tasks from the client.

5. Each of server should have a log file containing information about the queue number through time to visualize later.

3 Client

1. The client code will parse an input file containing a sequence of queries. Each line specifies a method name and an argument that should be invoked on the remote server. The input file will be provided at the course website. For each remote invocation, the client will print the result of the invocation and the time it took. The output should also be printed to a file. **The delay between each invocation is T milliseconds (T = [50, 20]).** This means that the next invocation must be delayed T milliseconds, regardless of whether the current invocation is finished or not.
2. Input file format: <method name> <arg1> <arg2> <arg3> <zone:#>
 - a. The method name describes one of the 4 interfaces that server provides. Depending on the method, it can take up to 3 arguments as input. The Zone:# describes the Zone of the Server to which the request must sent to.
 - b. eg:- getPopulationofCountry United States Zone:1
 - c. eg:- getNumberofCities Norway 568422 min Zone:4
 - d. eg:- getNumberofCountries 3 min 1616894 Zone:5
 - e. eg:- getNumberofCountriesMM 6 1677496 4406235 Zone:4
3. Output file format : <result> <input query> (turnaround time: YY ms, execution time: ZZ ms, waiting time: TT ms, processed by Server <server#>)
 - a. The result describes the result of the method invocation. The input query is the input query string. The third tuple describes the turnaround time, execution time and waiting time details along with the server information. (described in the next section)
 - b. eg:- : 9362428 getPopulationofCountry Sweden Zone:1 (turnaround time: 120 ms, execution time: 10 ms, waiting time: 100 ms, processed by Server 1)
 - c. eg:- 1 getNumberofCities Norway 568422 min Zone:3 (turnaround time: 150 ms, execution time: 10 ms, waiting time: 120 ms, processed by Server 4)
 - d. eg:- 17 getNumberofCountries 3 1616894 min Zone:5 (turnaround time: 100 ms, execution time: 10 ms, waiting time: 80 ms, processed by Server 5)
 - e. eg:- 2 getNumberofCountriesMM 6 1677496 4406235 Zone:2 (turnaround time: 120 ms, execution time: 10 ms, waiting time: 90 ms, processed by Server 4)
4. At the end of the output file, add 6 entries. Each entry should correspond to a type of method that server handled. Each entry should report the average turn-around time, execution time and waiting time.
 - a. <method name> avg turn-around time: <A> ms, avg execution time: ms, avg waiting time: <C> ms, min turn-around time: <D> ms, max turn-around time: <E> ms
5. Implementation tip: Define a data structure (e.g. a java class) to implement the tasks and to store all the information associated with them.

6. Note: The Client represents a simulator for set of clients. In real-world scenarios, there will be tens of thousands of clients that will send request to the servers. But **we use a single Client to represent all the clients of all zones.**

4 Docker/Container (For group of 4)

- The server execution program as well as its dependent libraries and tools must be containerized for deployment.
- You should be aware of the IP address of the deployed container. It can affect the registration of new server.

Summary of measurements:

- Turnaround time (from the moment a client gets to one of the servers in each zone until it receives the final response)
- Execution time (from the moment a server starts running the request until it finds the response)
- Waiting time (from the moment a server gets the request on waiting list until it starts to get executed)

Assumptions and Requirements

1 Naive Implementation

- The server parses the whole dataset every time a request is made. The server will have to read the complete dataset to answer the client's query.
- The client should name the output file as naive_server.txt. Each entry in the output file should correspond to a query from the input file

2 Server-Side Caching

- Each server maintains a cache. The cache can store up to 150 different results.
- If the information is not found on the cache, the naive implementation should be utilized to respond to the user query. Each server will have its own zone-specific cache independent from the servers at other zones
- The server only saves and updates the cache per request it receives in real-time. Meaning that at the very beginning the server cache is empty and the servers do not perform any pre-processing on the dataset!
- Cache will be updated dynamically and when the information that servers keep in cache exceeds the limit,
 - Method 1: They will release first entry from their memory and replace it with new information (FIFO)
 - Method 2: They will release the oldest recent used entry. Each entry should record the last timestamp it is being used and then find the oldest one (OLDEST).
- The client should name the output file as server_cache.txt. It is assumed that if the server contains the cache then the client also knows about it. The flags at the

command line can be used to enable cache in the server. The flags can also be used to notify the client that server contains cache.

3 Client-Side Caching

- The Client also maintains a cache. The cache can store the results returned by the Server. The Client-Side Cache can store up to 45 results.
- If the Client-Side Cache is enabled, then the Client should check the cache to respond to the query. If the information is not present in the cache, then it can send request to the Server.
- The Client can only save and update the cache per request it receives in real-time. Meaning that at the very beginning the cache is empty and the Client do not perform any pre-processing on the dataset!
- Cache will be updated dynamically and when the information that servers keep in cache exceeds the limit,
 - Method 1: They will release first entry from their memory and replace it with new information (FIFO)
 - Method 2: They will release the oldest recent used entry. Each entry should record the last timestamp it is being used and then find the oldest one (OLDEST).
- Report the same output as explained before in a file named “client_cache.txt”

4 Result:

- After finishing generating the output files, draw graphs:
 - The turn-around time of corresponding queries.
 - X-axis: queries number
 - Y-axis: turn-around time
 - Server graph (for each server):
 - X-axis: Unix timestamp
 - Y-axis: queue number at the corresponding timestamp
- Remember that there are two cases: $T=50$ and $T=20$.

5 Recommendation for splitting the workload:

- Member 1: Client and Proxy server
- Member 2: Processing Server, including queue technique
- Member 3: Cache technique
- Member 4: Docker/Container

Deliverables:

Via the Devilry system:

Your submission must be a compressed file containing:

- A description of your design and implementation (preferably PDF) including:
 - Screenshots of both client and server under execution.
 - An user guide for compiling, building, deploying and running your distributed application.

- Description of how you divided the workload among your group members.
- Everything that is needed to compile, build, deploy and run your distributed application (in a zip archive), such as:
 - Any artifact/file/library that is needed to compile and run your distributed application (in a zip archive)
 - Source code (**must be well commented** at method/algorithm granularity);
 - Ready to deploy jar file;
 - Ready to deploy docker image file; (If file size is too big, upload it to google drive or one drive and put the target URL to the submission file)
- Output files containing the required measurements: naive_server.txt, server_cache.txt, client_cache.txt.
- Graphs
- Any other artifact that you consider relevant to compile and run your application.

You can use any sorts of available libraries to develop your system. We encourage using Maven to import libraries into your projects. However, if you chose alternative ways to import libraries, you should also document how to do so.

Submission Deadline on devilry: 23:59 on September 24, 2026